University of Eloued
Faculty of Exact Sciences
Department of Computer science
For Second-year Master: IA & Data Science

University season: 2026/2027
Module: Big Data Analytics

TP04: Containerization and Big Data Environment Deployment Using Docker

Lab Report

Student: Bey Heyam

Docker Hub: beyheyam/tp04-analysis:v1

• 1. Work summary

The TP03 dataset (CMS Open Payments, general payments 2025) was cleaned in Power BI to the period January to March 2025, with duplicates and rows without a specialty removed. The same cleaning was reproduced in Python to export a CSV of 3,866,795 rows. A Python application reads this CSV in chunks, so the 9 GB source file never has to fit in memory, and it was packaged as a Docker image.

Task	What was done	Result
Task 1	Docker Desktop (WSL 2) installed, hello-world run, Docker Hub account created	docker --version OK; hello-world OK
Task 2	Image built from the TP03 dataset, pushed to Docker Hub, local copy removed, then pulled back	beyheyam/tp04-analysis:v1 (522 MB on disk)
Task 3	Network tp04-net and three containers c1, c2, c3 (ports 8001, 8002, 8003)	3 containers Up; same output in each log
Task 4	compose.yaml: build, run with ./data mounted read-only, then down	Exited (0); output identical

Output of the application (identical locally, in the container, and with Docker Compose):

Indicator	Value
Rows analysed	3,866,795
Total amount (USD)	641,735,348.00
Top 5 states (USD)	CA 71.59 M, FL 53.17 M, NY 50.71 M, TX 46.02 M, VA 42.82 M
Top 5 companies (USD)	Zimmer Biomet 40.87 M, AbbVie 32.84 M, Stryker 31.30 M, Medtronic 24.18 M, Arthrex 22.99 M

• 2. Docker Compose questions (Task 4)

What problem does Docker Compose solve?

Real applications usually need several services (application, database, cache) with shared networks and volumes. Starting each one with a long docker run command is slow and error-prone, and it is hard to reproduce on another machine. Docker Compose describes all services, networks, volumes and settings in one versioned file (compose.yaml) and starts the whole environment with a single command.

Compare docker run with docker compose up.

docker run is imperative: it creates one container, and every option (ports, volumes, network, name) is typed by hand in the command. docker compose up is declarative: it reads compose.yaml, builds the images if needed, creates the network and volumes, and starts every service in the right order. The result is reproducible, easy to share through Git, and easy to stop and clean with docker compose down.

How could you extend the Compose project to include a database service?

Add a second service that uses an official database image, give it a named volume so the data persists, and make the analysis service depend on it. The services reach each other by service name on the default Compose network (here the host name is db).

```
services:
  data-analysis:
    build: .
    depends_on: [db]
    environment:
      DB_HOST: db
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

• 3. Report (Task 5)

What is Docker, and why is it useful in data-science projects?

Docker is a platform that packages an application together with its libraries, runtime and configuration into a lightweight, isolated unit called a container. Containers share the host kernel, so they start fast and use fewer resources than virtual machines.

In data science it is useful because it:

- gives a reproducible environment: the same Python and library versions run on every machine, which removes the “works on my machine” problem;
- isolates projects, so conflicting dependencies do not interfere with each other;
- makes pipelines easy to share, version and deploy to a server or the cloud;
- allows big data tools (databases, Spark, Hadoop) to be installed and removed in minutes, and the same application to be scaled by running several containers.

Explain the difference between a Docker image and a container.

An image is a read-only template built from a Dockerfile, made of layers that contain the operating system files, the dependencies and the application code. A container is a running instance of an image: Docker adds a thin writable layer on top and starts a process. The same image can create many containers; in this lab, c1, c2 and c3 were all created from beyheyam/tp04-analysis:v1. A useful comparison is a class (image) and its objects (containers).

Explain the purpose of FROM, WORKDIR, COPY, RUN, and CMD in the Dockerfile.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "analysis.py"]
```

- **FROM**: chooses the base image on which the new image is built (here a small Python 3.12 image).
- **WORKDIR**: sets the working directory inside the image (/app); the following instructions and the container start from there.
- **COPY**: copies files from the host (the build context) into the image. Copying requirements.txt first lets Docker reuse the cached dependency layer when only the code changes.
- **RUN**: executes a command while the image is being built and saves the result as a layer (here pip install of the dependencies).
- **CMD**: defines the default command executed when a container starts (python analysis.py). It can be overridden at docker run, as was done in Task 3 to keep the containers alive.

• 4. Conclusion

Docker made the analysis portable and reproducible: the same image produced the same results on the local machine, in three independent containers, and through Docker Compose. Keeping the large CSV outside the image (read-only volume) kept the image small (522 MB) and fast to push to Docker Hub.